Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського» Кафедра цифрових технологій в енергетиці

Розрахунково-графічна робота

Тема: «Операції з текстурними координатами»

з дисципліни: «Візуалізація графічної та геометричної інформації»

Варіант 3

Виконала:
Студентка 1-го курсу магістратури
НН ІАТЕ
Групи ТР-62мп
Голуб Т.Д.

Київ – 2026

1. Опис завдання

Розрахунково-графічна робота спирається на результат контрольної роботи, у якій поверхню Зіверта було відображено з трьома текстурами: дифузною, що модулює дифузну складову освітлення, спекулярною, що модулює дзеркальну складову, та нормальною, яка зберігає нормаль для кожного пікселя. Нова функціональність стосується операцій над текстурними координатами.

Завдання(варіант 3): реалізувати масштабування текстури навколо точки, яку задає користувач. Текстура має збільшуватися або зменшуватися так, щоб ділянка поверхні в обраній точці лишалася на місці, а решта малюнка розтягувалася чи стискалася відносно неї. Точка масштабування обирається в просторі параметрів (u, v) поверхні, а не в екранних координатах, тому вона прив'язана до поверхні й рухається разом з нею під час обертання моделі. Переміщення точки виконується з клавіатури: клавіші A та D змінюють параметр u, а клавіші W та S змінюють параметр v. Для зручності користувача точка відображається на поверхні маркером.

2. Теоретичні відомості

2.1. Параметрична поверхня і текстурні координати

Поверхня Зіверта задається двома параметрами u та v. Для константи C = 2 її точка обчислюється за формулами:

$$\varphi = -u / \sqrt{(C+1)} + \operatorname{arctg}(\sqrt{(C+1)} \cdot \operatorname{tg} u),$$

$$a = 2 / (C + 1 - C \cdot \sin^2 v \cdot \cos^2 u),$$

$$r = (a / \sqrt{C}) \cdot \sqrt{((C+1)(1 + C \cdot \sin^2 u))} \cdot \sin v,$$

після чого

$$x = r \cdot \cos \varphi,$$

$$y = r \cdot \sin \varphi,$$

$$z = (\ln \operatorname{tg}(v/2) + a\,(C+1) \cos v) / \sqrt{C}.$$

Параметри змінюються в межах $u \in [-0{,}49\pi;\ 0{,}49\pi]$ та $v \in [0{,}02\pi;\ 0{,}98\pi]$, які трохи звужені, щоб уникнути ділення на нуль і логарифма нуля на краях.

Текстура є зображенням, до якого звертаються за текстурними

координатами (s, t) з одиничного квадрата. Щоб накласти її на поверхню, кожній вершині ставиться у відповідність пара (s, t), що лінійно залежить від параметрів: s = (u − u_min) / (u_max − u_min) та t = (v − v_min) / (v_max − v_min). Для внутрішніх точок трикутників координати інтерполюються апаратно, і кожен піксель отримує власну пару (s, t).

2.2. Масштабування текстурних координат навколо точки

Перетворення текстури виконується не над самим зображенням, а над координатами, за якими воно читається. Нехай p = (p_s, p_t) є точкою масштабування, а k > 0 коефіцієнтом. Тоді координати вибірки обчислюються так:

$$s' = p_s + (s - p_s) / k,\ t' = p_t + (t - p_t) / k.$$

У однорідних координатах це композиція трьох перетворень, M = T(p) · S(1/k) · T(−p): спочатку точка p переноситься в початок координат, потім виконується масштабування, а потім зсув повертається назад.

Ділення на k пояснюється так: щоб зображення виглядало у k разів більшим, кожен піксель на поверхні має читати ділянку текстури, яка в k разів менша. Тому при k > 1 текстура збільшується, при k < 1 зменшується, а при k = 1 перетворення тотожне. Точка p при цьому лишається нерухомою, бо для s = p_s маємо s' = p_s, і так само для t. Саме ця властивість відрізняє масштабування навколо заданої точки від масштабування навколо початку координат, при якому малюнок «з'їжджав» би в бік кута.

2.3. Режими вибірки і міпмапи

При k < 1 значення s' та t' виходять за межі відрізка від 0 до 1, тому треба визначити, що повертає текстура поза своїми межами. Режим CLAMP_TO_EDGE повторює крайній піксель і дає розмиті смуги, а режим REPEAT повторює зображення, що для цегляної кладки виглядає природно. У WebGL 1 режим REPEAT дозволений лише для текстур, сторони яких є степенями двійки. Використані в роботі текстури мають розмір 1024 на 1024 пікселі, тому ця умова виконується.

Для таких текстур додатково будується піраміда міпмап, яка містить

зменшені копії зображення. Вона потрібна, коли текстура зменшується (відображається меншою, ніж її розмір), бо без неї виникає мерехтіння і муар. Фільтрація LINEAR_MIPMAP_LINEAR вибирає два найближчих рівні піраміди й інтерполює між ними.

2.4. Нормальна карта і ортогоналізація Грама-Шмідта

Нормальна карта зберігає в кожному пікселі вектор нормалі в тангенційній системі координат. Кольорові компоненти c з діапазону від 0 до 1 перетворюються у вектор n_t = 2c − 1, компоненти якого лежать у діапазоні від −1 до 1. Щоб використати його в освітленні, потрібно перевести вектор у систему ока за допомогою матриці з трьох ортонормованих векторів: тангенса T, бітангенса B та нормалі N.

Інтерпольовані тангенс і нормаль не залишаються перпендикулярними, тому застосовується ортогоналізація Грама-Шмідта. За варіантом 3 пріоритет має тангенс: він лишається без змін, а нормаль очищається від складової вздовж нього, N' = нормалізувати(N − (N · T) · T). Бітангенс знаходиться як векторний добуток B = N' × T, і нормаль у системі ока дорівнює n = [T B N'] · n_t.

Рівномірне масштабування текстурних координат змінює лише те, з якого місця карти береться вектор, і не повертає самих векторів. Тому нормалі, збережені в карті, лишаються правильними, і додаткової корекції тангенційного базису для варіанта з масштабуванням не потрібно.

2.5. Модель освітлення

Освітлення обчислюється за моделлю Фонга в кожному пікселі як сума трьох складових, I = I_a + I_d + I_s. Фонова складова дорівнює 0,2 · k_d, дифузна дорівнює max(N · L, 0) · 0,8 · k_d, дзеркальна дорівнює max(R · V, 0)^40 · 0,6 · k_s. Тут L є напрямом на джерело світла, R відбитим променем, V вектором на глядача, взятим як (0, 0, 1), а k_d та k_s кольорами з дифузної та спекулярної текстур, прочитаними за перетвореними координатами (s', t'). Завдяки цьому три текстури масштабуються синхронно, і колір, блік та рельєф лишаються узгодженими.

3. Особливості реалізації

Застосунок складається з файлів index.html, main.js, Model.js, TextureHandler.js і shader.gpu, а також з папки textures із трьома зображеннями. Стан перетворення зберігається в трьох змінних: texPointU та texPointV для положення точки і texScale для коефіцієнта. Точка зберігається в нормованих параметрах від 0 до 1, початкові значення дорівнюють 0,5 та 0,5, а масштаб 1. Нормовані параметри збігаються з текстурними координатами, бо в Model.js текстурні координати вершини з індексами i та j дорівнюють i / numU та j / numV, тобто лінійно відповідають u та v. Тому положення точки не потрібно додатково перераховувати.

Керування точкою виконує обробник події keydown, підключений до вікна. Один крок дорівнює 0,02, а після зміни значення обмежуються відрізком від 0 до 1 і оновлюється підпис з поточними координатами точки. Масштаб змінюється окремим слайдером у діапазоні від 0,2 до 4 з кроком 0,05.

У функції draw кожен кадр у шейдер передаються uniform-змінні texPoint та texScale. У фрагментному шейдері першим рядком обчислюються перетворені координати tc = texPoint + (vTex − texPoint) / texScale, і саме tc використовується для читання всіх трьох текстур. Оскільки перетворення афінне, його можна було б виконати й у вершинному шейдері, бо лінійна інтерполяція дала б той самий результат, але в фрагментному шейдері воно записується одним рядком поруч з іншими розрахунками і зручніше для пояснення.

Для наочності положення точки на поверхні відображається маркером. Метод PointAt у Model.js приймає нормовані параметри й повертає координати точки поверхні, викликаючи ту саму функцію surfacePoint, що й побудова сітки. Маркер малюється червоним кольором окремою невеликою шейдерною програмою, яка використовується і для жовтого маркера джерела світла. Щоб маркер не перекривала сама поверхня, у вершинному шейдері маркера z-координата трохи зсувається до камери.

У файлі TextureHandler.js після завантаження зображення для текстур зі сторонами у степенях двійки генеруються мипмапи і вмикається режим

REPEAT. Решта функціональності контрольної роботи збережена: слайдери деталізації сітки за U та V, нормалі, зважені за кутом, ортогоналізація з пріоритетом тангенса, світло, що обертається по колу, та розмір полотна, який підлаштовується під вікно разом із відповідним співвідношенням сторін у матриці проєкції.

4. Інструкція користувача

4.1. Запуск застосунку

Відкрийте в редакторі VS Code папку проєкту та клацніть правою кнопкою миші на index.html та виберіть Open with Live Server або натисніть кнопку Go Live в рядку стану. Браузер відкриє сторінку за адресою на кшталт http://127.0.0.1:5500/index.html (рисунок 1).

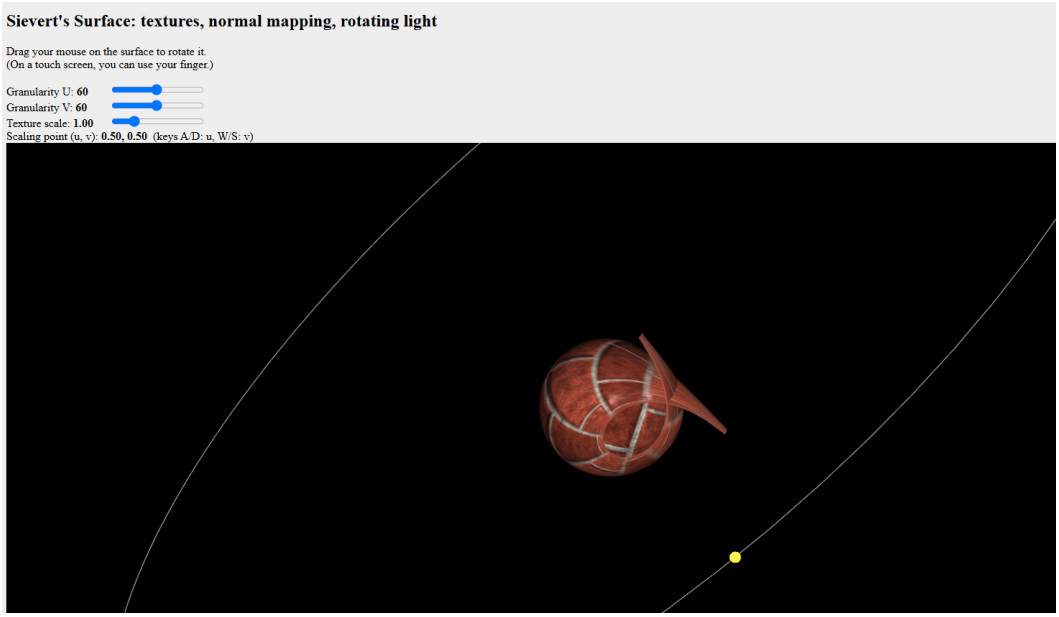


Рисунок 1 – Загальний вигляд застосунку після запуску

4.2. Елементи керування

Поверхню можна обертати, перетягуючи її мишею. Два слайдери Granularity U та Granularity V незалежно задають кількість розбиттів сітки за параметрами u та v, тому збільшення одного з них згладжує поверхню лише в одному напрямку. Слайдер Texture scale задає коефіцієнт масштабування

текстури від 0,2 до 4. Під слайдерами виводиться рядок Scaling point (u, v) з поточними нормованими координатами точки масштабування, а на самій поверхні ця точка показана червоним маркером. Жовта точка і сіре коло показують положення та траєкторію джерела світла, яке безперервно обертається навколо поверхні.

4.3. Масштабування текстури

Щоб змінити масштаб, перетягніть слайдер Texture scale. Для значень більших за одиницю цегляна текстура збільшується, а для значень менших за одиницю зменшується й повторюється. Місце під червоною точкою при цьому лишається на місці, а решта малюнка розтягується або стискається відносно нього. Усі три текстури масштабуються разом, тож кольори цегли, блік і рельєф швів лишаються узгодженими.

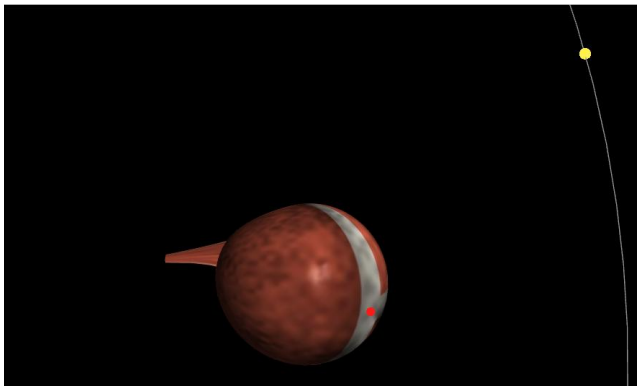


Рисунок 2 – Збільшення текстури (масштаб більший за 1)

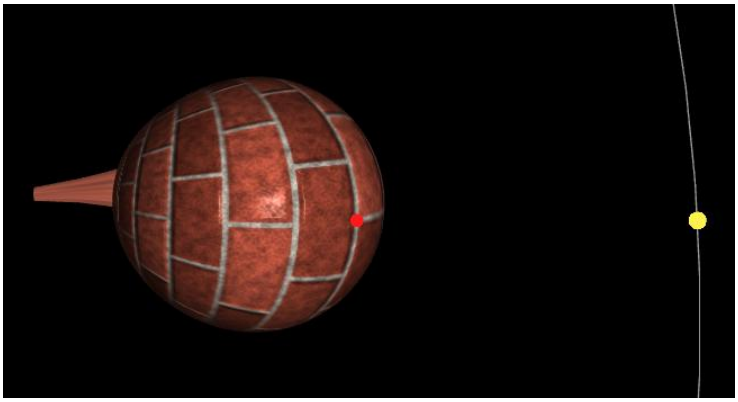


Рисунок 3 – Зменшення текстури (масштаб менший за 1)

4.4. Переміщення точки масштабування

Точка рухається клавішами: A і D змінюють параметр u (ліворуч і праворуч), W і S змінюють параметр v (вгору і вниз). Один натиск зсуває точку на 0,02 від повного діапазону, а якщо клавішу утримувати, точка рухається безперервно. Клавіші працюють незалежно від розкладки клавіатури.

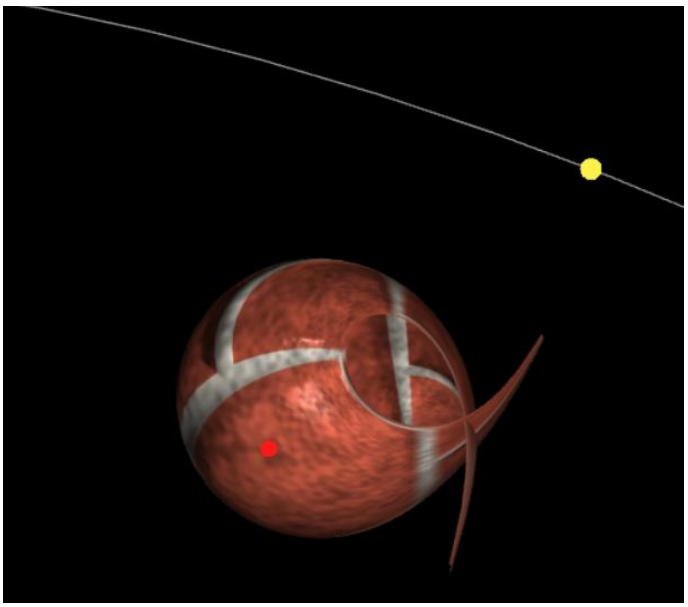


Рисунок 4 – Масштабування навколо точки, зсунутої від центру

5. Фрагменти вихідного коду

5.1. Фрагментний шейдер (shader.gpu)

У шейдері обчислюються перетворені текстурні координати tc та використовуються для всіх трьох текстур. Тут же виконується ортогоналізація Грама-Шмідта з пріоритетом тангенса.

```
uniform vec2 texPoint;       // точка масштабування в (u,v), [0;1]
uniform float texScale;      // коефіцієнт масштабування текстури

varying vec3 vPos;
varying vec3 vNormal;
varying vec3 vTangent;
```

```
varying vec2 vTex;

void main() {
    // Масштабування текстурних координат навколо точки texPoint
    vec2 tc = texPoint + (vTex - texPoint) / texScale;

    // Gram-Schmidt з пріоритетом тангенса
    vec3 T = normalize(vTangent);
    vec3 N = normalize(vNormal);
    N = normalize(N - dot(N, T) * T);
    vec3 B = cross(N, T);
    mat3 TBN = mat3(T, B, N);

    vec3 nt = texture2D(normalMap, tc).rgb * 2.0 - 1.0;
    vec3 Nm = normalize(TBN * nt);

    vec3 V = vec3(0.0, 0.0, 1.0);
    if (dot(Nm, V) < 0.0) Nm = -Nm;

    vec3 L = normalize(lightPos - vPos);
    vec3 R = reflect(-L, Nm);

    vec3 diffTex = texture2D(diffuseMap, tc).rgb;
    vec3 specTex = texture2D(specularMap, tc).rgb;

    vec3 ambient  = 0.2 * diffTex;
    vec3 diffuse  = max(dot(Nm, L), 0.0) * 0.8 * diffTex;
    vec3 specular = pow(max(dot(R, V), 0.0), 40.0) * 0.6 * specTex;

    gl_FragColor = vec4(ambient + diffuse + specular, 1.0);
}`;
```

5.2. Керування точкою та масштабом (main.js)

```
let texPointU = 0.5;        // точка масштабування в (u,v), нормована до [0;1]
let texPointV = 0.5;
let texScale = 1.0;         // коефіцієнт масштабування
const POINT_STEP = 0.02;    // крок руху точки за одне натискання


// A/D змінюють u, W/S змінюють v (e.code не залежить від розкладки)
function onKeyDown(e) {
  switch (e.code) {
```

```
    case 'KeyA': texPointU -= POINT_STEP; break;
    case 'KeyD': texPointU += POINT_STEP; break;
    case 'KeyW': texPointV += POINT_STEP; break;
    case 'KeyS': texPointV -= POINT_STEP; break;
    default: return;
  }
  texPointU = Math.min(1, Math.max(0, texPointU));
  texPointV = Math.min(1, Math.max(0, texPointV));
  updatePointLabel();
}

function updateScale() {
  texScale = parseFloat(document.getElementById('sliderScale').value);
  document.getElementById('valScale').textContent = texScale.toFixed(2);
}

// у функції draw(), кожен кадр:
gl.uniform2f(shProgram.iTexPoint, texPointU, texPointV);
gl.uniform1f(shProgram.iTexScale, texScale);
// ...
surface.Draw();
const texPointModel = surface.PointAt(texPointU, texPointV);
drawMarker(modelViewProjection, lightModel, texPointModel);
```

5.3. Текстурні координати та точка на поверхні (Model.js)

```
      this.BufferData = function (numU, numV) {
          const vertices = [];
          const tangents = [];
          const texCoords = [];
          const indices = [];

          for (let i = 0; i <= numU; i++) {
            const u = uMin + (i / numU) * (uMax - uMin);
            for (let j = 0; j <= numV; j++) {
              const v = vMin + (j / numV) * (vMax - vMin);
              vertices.push(...surfacePoint(u, v));
              tangents.push(...surfaceTangent(u, v));
              texCoords.push(i / numU, j / numV);   // текстура розтягується на всю
поверхню
            }
          }
```

```
          const idx = (i, j) => i * (numV + 1) + j;
          for (let i = 0; i < numU; i++) {
            for (let j = 0; j < numV; j++) {
              indices.push(idx(i, j), idx(i + 1, j), idx(i, j + 1));
              indices.push(idx(i + 1, j), idx(i + 1, j + 1), idx(i, j + 1));
            }
          }

          const normals = angleWeightedNormals(vertices, indices);
          this.count = indices.length;

          gl.bindBuffer(gl.ARRAY_BUFFER, this.vBuffer);
          gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices),
gl.STATIC_DRAW);

          gl.bindBuffer(gl.ARRAY_BUFFER, this.nBuffer);
          gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(normals), gl.STATIC_DRAW);

          gl.bindBuffer(gl.ARRAY_BUFFER, this.tBuffer);
          gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(tangents),
gl.STATIC_DRAW);

          gl.bindBuffer(gl.ARRAY_BUFFER, this.tcBuffer);
          gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(texCoords),
gl.STATIC_DRAW);

          gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.iBuffer);
          gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint16Array(indices),
gl.STATIC_DRAW);
        };

          // Точка поверхні за нормованими параметрами pu, pv з [0;1] (для маркера)
        this.PointAt = function (pu, pv) {
          return surfacePoint(uMin + pu * (uMax - uMin), vMin + pv * (vMax - vMin));
        };
      ...
```

5.4. Режим повторення текстури (TextureHandler.js)

```
      const isPow2 = (x) => (x & (x - 1)) === 0;
          if (isPow2(image.width) && isPow2(image.height)) {
            gl.generateMipmap(gl.TEXTURE_2D);
            gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
gl.LINEAR_MIPMAP_LINEAR);
```

```
            gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);
            gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);
        }
```

Посилання на github: https://github.com/taniaholub/VGGI.git